

Conceptual Model — EMS Discrete-Event Simulation

Document version: 1.0
Date: March 12, 2026
Project: EMS Readiness Optimization — Manhattan, NYC
Phase: 4 (Simulation Design)
Status: Approved for implementation

Table of Contents

- 1. [System Description](#)
 - 2. [Entities](#)
 - 3. [Resources](#)
 - 4. [Queues](#)
 - 5. [Events](#)
 - 6. [State Variables](#)
 - 7. [Dispatch Logic](#)
 - 8. [Performance Measures](#)
 - 9. [Time Representation](#)
 - 10. [Random Phenomena](#)
 - 11. [Event Flow Diagram](#)
 - 12. [Assumptions and Limitations](#)
 - 13. [References](#)
-

1. System Description

1.1 Purpose

This simulation evaluates **strategic EMS unit staging policies** under stochastic crash-driven demand in Manhattan, New York City. The central research question is:

Given K ambulance units and 48 candidate firehouses, which allocation policy minimises average response time and maximises the fraction of incidents served within an 8-minute threshold?

Five candidate policies are compared:

Code	Policy	Source
P0	Uniform allocation	Baseline heuristic
P1	Demand-proportional allocation	Baseline heuristic
P2	Demand-weighted optimisation	MIP (Phase 3)
P2-alt	P-median selection	MIP (Phase 3)
P2-cov	Maximal coverage	MIP (Phase 3)

1.2 Study Type — Terminating Simulation

The simulation is classified as a **terminating** (finite-horizon) simulation rather than a steady-state simulation.

Criterion	Rationale
Natural endpoint	Each replication covers a fixed horizon (default 168 hours = 1 week). The system has a well-defined start (empty-and-idle) and end.
Initial conditions	At $t = 0$ every EMS unit is available at its assigned firehouse and no incidents are in queue. This is a realistic depiction of a fresh deployment cycle.
Cyclic non-stationarity	Crash demand follows a Non-Homogeneous Poisson Process with pronounced hourly and day-of-week patterns (λ varies by factor $\approx 0.3\text{--}2.1\times$ from base rate 3.48 /hr). Steady-state assumptions require stationarity, which does not hold here.
Policy evaluation horizon	Decision-makers evaluate staging plans on a weekly operational cycle, matching the 168-hour horizon.
Replication strategy	Statistical inference is drawn from R independent replications (default $R = 30$), each with a different random-number stream, following the method of independent replications for terminating simulations (Law 2015, Ch. 9).

Because the simulation is terminating, there is **no warm-up period** to discard. Performance measures are computed over the entire horizon of each replication.

1.3 System Boundary

The system boundary encompasses:

- **Included:** Crash incident arrivals, EMS unit dispatch, travel to scene, on-scene service, return-to-available transition, queueing of unserved incidents.
 - **Excluded:** Real-time unit relocation, congestion-dependent travel, full road-network routing, dispatch priority / preemption, hospital turnaround delays, non-crash EMS calls.
-

2. Entities

2.1 Incident (Temporary Entity)

An **Incident** represents a motor-vehicle crash event that requires exactly one EMS unit for service. Incidents are created dynamically by the arrival process and destroyed after service completion.

Attribute	Type	Description
<code>incident_id</code>	int	Unique sequential identifier
<code>arrival_time</code>	float	Simulation clock time of occurrence (hours)
<code>precinct</code>	int	NYPD precinct where the crash occurs (spatial allocation)
<code>location_lat</code>	float	Latitude of incident (precinct centroid proxy)
<code>location_lon</code>	float	Longitude of incident (precinct centroid proxy)
<code>assigned_unit</code>	str / None	ID of the EMS unit assigned (None while queued)
<code>dispatch_time</code>	float	Clock time when a unit is dispatched
<code>service_start_time</code>	float	Clock time when unit arrives on scene
<code>service_end_time</code>	float	Clock time when unit completes service
<code>dispatch_delay</code>	float	<code>dispatch_time</code> - <code>arrival_time</code> (wait in queue + fixed processing)
<code>travel_time</code>	float	Travel time from unit's current location to incident (minutes)
<code>service_time</code>	float	On-scene service duration (minutes)
<code>response_time</code>	float	<code>dispatch_delay</code> + <code>travel_time</code> (total from crash to unit arrival on scene)

2.2 Entity Lifecycle

Created → Queued (if no unit available) → Dispatched → In Service → Completed (destroyed)

3. Resources

3.1 EMS Unit (Permanent Entity)

EMS Units are the **servers** in this system. They are stationed at FDNY firehouses according to the allocation policy under evaluation and remain in the system for the entire simulation horizon.

Attribute	Type	Description
<code>unit_id</code>	str	Unique identifier (e.g., <code>E026_1</code> for unit 1 at Engine 26)
<code>home_firehouse</code>	str	Assigned firehouse (Facility-Name)
<code>home_lat</code>	float	Latitude of home firehouse
<code>home_lon</code>	float	Longitude of home firehouse
<code>current_lat</code>	float	Current latitude (home when idle, incident location when busy)
<code>current_lon</code>	float	Current longitude
<code>status</code>	enum	<code>AVAILABLE</code> · <code>DISPATCHED</code> · <code>ON_SCENE</code> · <code>RETURNING</code>
<code>busy_since</code>	float / None	Clock time when unit last became busy
<code>total_busy_time</code>	float	Cumulative busy time (for utilisation calculation)
<code>calls_served</code>	int	Count of incidents served

3.2 Capacity

Total fleet size **K** is a scenario parameter (default values: 20, 30, 40, 48).

Units are distributed across 48 Manhattan firehouses subject to:

- Per-firehouse capacity cap: **C = 2** units maximum (default; sensitivity analysis tested C = 1-5).
- Allocation vector $\mathbf{x} = (x_1, x_2, \dots, x_{48})$ where $\sum x_i = K$ and $0 \leq x_i \leq C$.

The allocation vector is determined **before** the simulation begins and remains **fixed** throughout the replication (static staging policy).

3.3 Firehouse Locations

48 Manhattan FDNY firehouses serve as candidate staging locations. Their coordinates are loaded from `data/raw/FDNY_Firehouse_Listing_20260223.csv`. A pre-computed distance matrix (`data/processed/distance_matrix_firehouse_precinct.csv`) gives Haversine distances between each firehouse and each of the 30 Manhattan precinct centroids.

4. Queues

4.1 Waiting Incidents Queue

Property	Specification
Name	<code>incident_queue</code>
Discipline	First-In, First-Out (FIFO)
Capacity	Unlimited (no balking or reneging)
Trigger	An arriving incident enters the queue when all EMS units across all firehouses are currently busy.
Drain	When a unit completes service and becomes available, the longest-waiting incident (queue head) is immediately dispatched.
Tracked statistics	Queue length over time, maximum queue length, time-weighted average queue length, individual waiting times.

4.2 Queueing Dynamics

The system is analogous to a spatially distributed **M(t)/G/K** queue:

- **M(t)**: Non-Homogeneous Poisson arrivals (time-varying rate).
- **G**: General (LogNormal) service-time distribution.
- **K**: Total EMS units across all firehouses.

Unlike a classical queue, travel time depends on the dispatched unit's location relative to the incident, introducing a spatial component absent from standard queueing models. This is a key motivation for using simulation rather than analytical queueing formulae.

5. Events

The simulation is driven by five event types. Each is described with its triggering condition, detailed logic, and state transitions.

5.1 Incident Arrival

Property	Detail
Trigger	Generated by the NHPP thinning algorithm at rate $\lambda(t)$.
Frequency	Base rate 3.48 /hr $\times$ hourly factor $\times$ day-of-week factor.
Logic	

```

INCIDENT_ARRIVAL(t):
  1. Create new Incident entity with arrival_time = t
  2. Assign precinct via multinomial draw from precinct probabilities
  3. Set incident location to precinct centroid (lat, lon)
  4. IF any EMS unit has status == AVAILABLE:
    → trigger DISPATCH_DECISION(incident)
  ELSE:
    → enqueue incident in incident_queue (FIFO)
  5. Schedule next INCIDENT_ARRIVAL via thinning algorithm

```

5.2 Dispatch Decision

Property	Detail
Trigger	(a) New incident arrives and ≥ 1 unit is available, OR (b) a unit completes service and the queue is non-empty.
Logic	

```

DISPATCH_DECISION(incident):
  1. Compute travel time from each AVAILABLE unit to incident location:
    t_travel(u, inc) = haversine(u.current_lat, u.current_lon,
    inc.location_lat, inc.location_lon)
    / effective_speed(hour_of_day)
  2. Select unit  $u^* = \operatorname{argmin} \{ t\_travel(u, inc) : u \in \text{AVAILABLE} \}$ 
    Tie-breaking: lowest unit_id (deterministic)
  3. Set  $u^*.status = \text{DISPATCHED}$ 
  4. Set incident.assigned_unit =  $u^*.unit\_id$ 
  5. Set incident.dispatch_time = current_time
  6. Add fixed dispatch processing delay  $\delta = 1.5$  minutes
  7. Compute travel_time = t_travel( $u^*$ , inc)
  8. Schedule SERVICE_START at  $t + \delta/60 + \text{travel\_time}/60$ 
    (all converted to hours for the simulation clock)

```

5.3 Service Start

Property	Detail
Trigger	Scheduled by DISPATCH_DECISION after dispatch delay + travel time elapses.
Logic	

```
SERVICE_START(incident, unit):
1. Set unit.status = ON_SCENE
2. Set unit.current_lat = incident.location_lat
3. Set unit.current_lon = incident.location_lon
4. Set incident.service_start_time = current_time
5. Draw service_duration ~ LogNormal( $\mu=3.136$ ,  $\sigma=0.385$ )
   [moment-matched: mean=25 min, std=10 min]
6. Schedule SERVICE_COMPLETION at current_time + service_duration/60
```

5.4 Service Completion

Property	Detail
Trigger	Scheduled by SERVICE_START after service duration elapses.
Logic	

```
SERVICE_COMPLETION(incident, unit):
1. Set incident.service_end_time = current_time
2. Record incident performance metrics
3. Set unit.status = AVAILABLE
4. Set unit.current_lat = unit.home_lat (return to firehouse)
5. Set unit.current_lon = unit.home_lon
6. Update unit.total_busy_time += (current_time - unit.busy_since)
7. Increment unit.calls_served
8. IF incident_queue is NOT empty:
   → dequeue oldest incident
   → trigger DISPATCH_DECISION(dequeued_incident)
```

Note on return travel: The unit is modelled as returning to its home firehouse **instantaneously** upon service completion. This is a simplifying assumption; the service-time distribution is calibrated to absorb the return component (see § 12, Assumption A-7).

5.5 End of Simulation

Property	Detail
Trigger	Simulation clock reaches the horizon T (default 168 hours).
Logic	

```
END_OF_SIMULATION(T):
1. Stop generating new incident arrivals
2. Allow in-progress services to complete (drain phase)
3. For any incidents still in queue: mark as "unserved"
4. Compute replication-level summary statistics:
  a. Average dispatch delay
  b. Average response time
  c. Fraction of incidents with response_time ≤ T (8 min)
  d. Maximum and mean queue length
  e. Per-unit utilisation = total_busy_time / T
  f. Total incidents served / arrived / unserved
5. Return replication results
```

6. State Variables

State variables define the complete system state at any point in simulated time. They are updated exclusively by the event routines described in § 5.

6.1 Core State

Variable	Type	Description
<code>sim_clock</code>	float	Current simulation time (hours from $t = 0$)
<code>units</code>	<code>list[EMS_Unit]</code>	All EMS unit objects with current attributes
<code>available_units</code>	<code>set[str]</code>	Set of unit IDs with status <code>AVAILABLE</code>
<code>incident_queue</code>	<code>deque[Incident]</code>	FIFO queue of waiting incidents
<code>active_incidents</code>	<code>dict[int, Incident]</code>	Incidents currently being served
<code>completed_incidents</code>	<code>list[Incident]</code>	All completed incident records

6.2 Statistical Accumulators

Accumulator	Type	Updated By
total_dispatch_delay	float	SERVICE_START
total_response_time	float	SERVICE_START
total_service_time	float	SERVICE_COMPLETION
count_within_threshold	int	SERVICE_START (if response_time $\leq \tau$)
count_delayed	int	DISPATCH_DECISION (if dispatch_delay > 0)
max_queue_length	int	INCIDENT_ARRIVAL / SERVICE_COMPLETION
queue_length_area	float	Time-weighted area under queue-length curve
incidents_arrived	int	INCIDENT_ARRIVAL
incidents_served	int	SERVICE_COMPLETION

6.3 Configuration Parameters

Parameter	Symbol	Default	Source
Simulation horizon	T	168 h (1 week)	configs/demand.yaml
Total EMS units	K	40	configs/optimiza- tion.yaml
Per-firehouse capa- city	C	5	configs/optimiza- tion.yaml
Base arrival rate	λ_0	3.48 /hr	configs/demand.yaml
Average EMS speed	v	20 mph	configs/ser- vice.yaml
Service time mean	μ_s	25 min	configs/ser- vice.yaml
Service time std	σ_s	10 min	configs/ser- vice.yaml
Dispatch processing delay	δ	1.5 min	configs/ser- vice.yaml
Response-time threshold	τ	8 min	configs/optimiza- tion.yaml
Replications	R	30	configs/demand.yaml
Random seed	—	42	configs/demand.yaml

7. Dispatch Logic

7.1 Nearest-Available Dispatch

The dispatch policy is **nearest-available**: the available unit with the shortest estimated travel time to the incident location is selected.

Travel-time estimate:

$$t_{\text{travel}}(u, j) = \frac{d_{\text{haversine}}(u_{\text{loc}}, j_{\text{loc}})}{v_{\text{eff}}(h)}$$

where:

- $d_{\text{haversine}}$ is the great-circle distance in miles,
- $v_{\text{eff}}(h) = v_{\text{base}} \times f_{\text{TOD}}(h)$ is the effective speed at hour h ,

- f_{TOD} is the time-of-day speed multiplier from `configs/service.yaml`.

7.2 Tie-Breaking Rule

When multiple available units have identical minimum travel times (e.g., two units co-located at the same firehouse), the unit with the **lowest unit ID** (lexicographic order) is selected. This provides deterministic, reproducible behaviour.

7.3 Queue Discipline

- **FIFO:** When a unit becomes available and the queue is non-empty, the incident that has been waiting the longest (head of the deque) is dispatched first.
- **No priority classes:** All incidents are treated equally regardless of severity. Priority dispatch is listed as out-of-scope.
- **No preemption:** Once a unit begins serving an incident, it cannot be recalled for a higher-priority call.

7.4 Dispatch upon Service Completion

When a unit completes service and returns to the available pool, the system checks the queue before the unit becomes idle. If incidents are waiting, dispatch occurs immediately—there is no idle gap between consecutive services when the queue is non-empty.

8. Performance Measures

All measures are computed per replication and then summarised across R replications using point estimates and 95% confidence intervals.

8.1 Primary Measures

Measure	Formula	Units
Average dispatch delay	$\bar{W}_q = \frac{1}{N} \sum_{i=1}^N (d_{i,\text{dispatch_time}} - d_{i,\text{arrival_time}})$	minutes
Average response time	$\bar{R} = \frac{1}{N} \sum_{i=1}^N (d_{i,\text{dispatch_delay}} + d_{i,\text{travel_time}})$	minutes
Coverage (% within τ)	$P(\text{resp} \leq \tau) = \frac{1}{N} \sum_{i=1}^N \mathbb{1}_{[R_i \leq \tau]}$	%

8.2 Secondary Measures

Measure	Formula	Units
Mean queue length	$\bar{L}_q = \frac{1}{T} \int_0^T Q(t) dt$ (time-weighted)	incidents
Maximum queue length	$\max_{t \in [0, T]} Q(t)$	incidents
Unit utilisation	$\rho_u = \frac{\text{busy_time}_u}{T}$, system avg: $\bar{\rho} = \frac{1}{K} \sum \rho_u$	fraction
Number of delayed incidents	$N_{\text{delayed}} = \sum \mathbb{1}[W_{q,i} > 0]$	count
Total incidents served	N_{served}	count
Fraction unserved at horizon	$N_{\text{unserved}} / N_{\text{arrived}}$	fraction

8.3 Statistical Output Analysis

For each performance measure Y , across R independent replications:

- Point estimate: $\bar{Y} = \frac{1}{R} \sum_{r=1}^R Y_r$
 - Standard error: $SE = s_Y / \sqrt{R}$
 - 95% confidence interval: $\bar{Y} \pm t_{R-1, 0.025} \cdot SE$
 - Comparison across policies via paired-t or Welch's t-test on replication-level means using Common Random Numbers (CRN).
-

9. Time Representation

9.1 Simulation Clock

Property	Specification
Time model	Continuous (real-valued)
Units	Hours from simulation start ($t = 0$)
Advancement	Next-event time advance (SimPy process-based)
Framework	SimPy 4.x (Python process-interaction DES library)

The simulation clock does **not** advance in fixed increments. Instead, it jumps from one scheduled event to the next, making it efficient even for long horizons with variable inter-event gaps.

9.2 Simulation Horizon

Parameter	Default	Notes
Horizon T	168 hours	1 full week (Mon 00:00 – Sun 23:59)
Start hour	0 (Monday midnight)	Configurable; aligns with DOW factor indexing
Configurable?	Yes	Via <code>configs/demand.yaml</code> → <code>simulation.default_duration_hours</code>

The 168-hour (1-week) horizon captures the full weekly demand cycle, including weekday/weekend variation and all hourly peaks.

9.3 Time Conversions

All internal calculations use **hours** as the canonical unit. When service times and travel times are drawn in minutes, they are divided by 60 before being added to the simulation clock.

10. Random Phenomena

10.1 Incident Arrivals — Non-Homogeneous Poisson Process (NHPP)

Property	Specification
Distribution	Non-Homogeneous Poisson Process
Algorithm	Lewis-Shedler thinning (acceptance-rejection)
Base rate	$\lambda_0 = 3.48$ crashes / hour (Manhattan-wide)
Rate function	$\lambda(t) = \lambda_0 \times f_hour(h(t)) \times f_dow(d(t))$
Hourly factors	Loaded from <code>data/processed/demand_lambda_hourly.csv</code>
DOW factors	Loaded from <code>data/processed/demand_lambda_dow.csv</code>
Spatial allocation	Multinomial draw over 30 precincts, probabilities from <code>data/processed/demand_lambda_precinct.csv</code>
Envelope rate	$\lambda_max = \lambda_0 \times \max(f_hour) \times \max(f_dow)$

Thinning Algorithm Summary:

1. Compute λ_max over the simulation horizon
2. Generate candidate arrival from `Exponential(1/ $\lambda\_max$ )`
3. Advance clock **to** candidate time t_c
4. Compute acceptance probability $p = \lambda(t_c) / \lambda_max$
5. Draw $U \sim \text{Uniform}(0,1)$
6. **IF** $U \leq p$: accept arrival; assign precinct; record event
ELSE: reject (thin) the candidate
7. Repeat from **step** 2 until $t_c > T$

10.2 Service Times — LogNormal Distribution

Property	Specification
Distribution	LogNormal(μ , σ)
Target moments	Mean = 25 min, Std = 10 min
LogNormal parameters	$\mu = \ln(25^2 / \sqrt{(25^2 + 10^2)}) \approx 3.136$, $\sigma = \sqrt{(\ln(1 + (10/25)^2))} \approx 0.385$
Support	$(0, \infty)$ — strictly positive
Justification	Right-skewed, positive, matches empirical EMS service-time profiles (see <code>docs/service_model_spec.md</code>)

10.3 Travel Times — Deterministic Given State

Travel times are **not** random draws. They are deterministic functions of:

- The dispatched unit's current location (lat, lon),
- The incident location (precinct centroid),
- The Haversine distance between them,
- The effective speed at the current hour of day.

Stochasticity in travel time arises **indirectly** through (a) random incident locations (precinct assignment) and (b) which unit happens to be available (depends on prior random service completions).

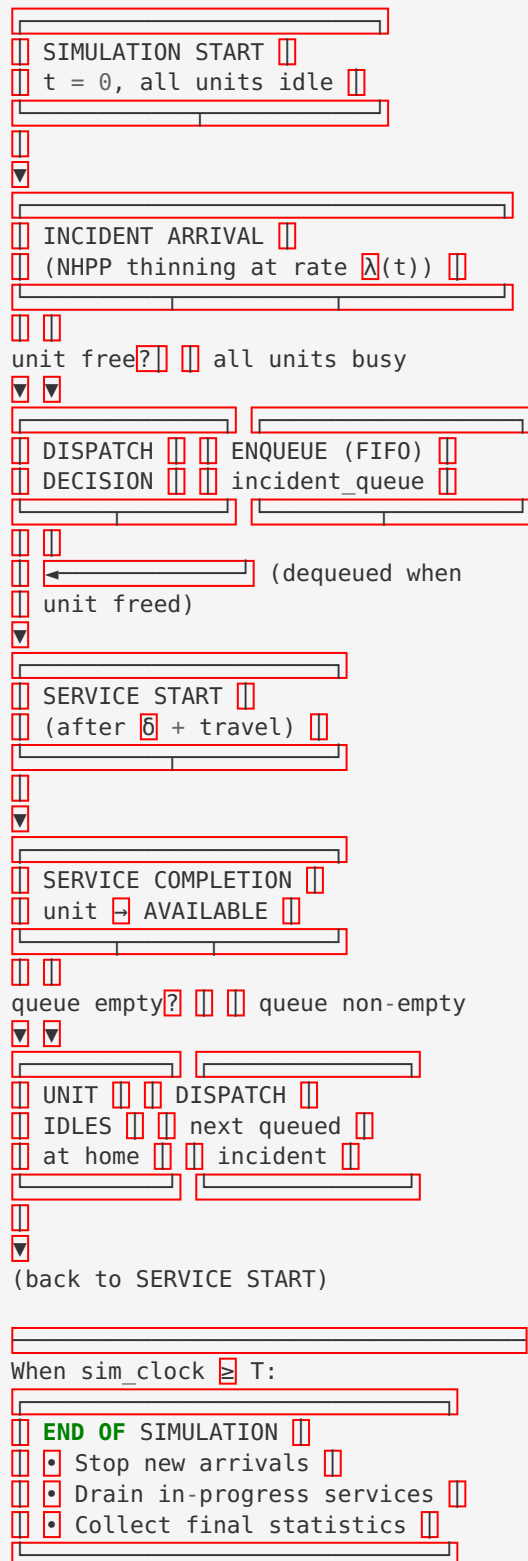
10.4 Random Number Streams and Reproducibility

Stream	Purpose	Seed Strategy
Stream 1	NHPP arrival generation (thinning)	Base seed + replication index
Stream 2	Precinct assignment (multinomial)	Base seed + replication index + offset
Stream 3	Service-time sampling (LogNormal)	Base seed + replication index + offset

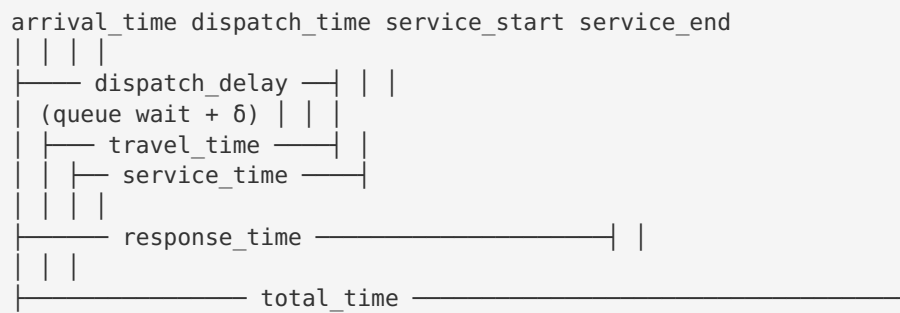
- **Base seed:** 42 (configurable in `configs/demand.yaml`).
 - **Dedicated streams** ensure that changes to one random component (e.g., service time distribution) do not alter the arrival sequence, enabling valid variance-reduction via **Common Random Numbers (CRN)**.
 - NumPy `Generator` (PCG64) is used for all sampling.
-

11. Event Flow Diagram

11.1 High-Level Event Transition Diagram



11.2 Single-Incident Timeline



11.3 Process-Interaction View (SimPy)

```

ArrivalProcess (SimPy generator) |
while sim_clock < T: |
yield env.timeout(inter_arrival) ← thinning |
if accepted: |
env.process(IncidentProcess(incident)) |

IncidentProcess (SimPy generator, one per incident) |
request unit from resource pool |
yield request ← may wait in queue |
yield env.timeout(dispatch_delay + travel_time) |
yield env.timeout(service_time) |
release unit back to resource pool |
record all timestamps |

```

12. Assumptions and Limitations

12.1 Key Assumptions

ID	Assumption	Justification	Impact if Violated
A-1	Historical crash data (2013–2026) is representative of future demand patterns.	Long time-series provides stable hourly/DOW/precinct estimates.	Demand mis-specification; mitigated by sensitivity analysis on λ_o .
A-2	Each incident requires exactly one EMS unit.	Simplifies dispatch; most crashes require a single ambulance.	Under-estimates resource demand for multi-unit incidents.
A-3	Travel distance approximated by Haversine (great-circle).	Avoids road-network dependency; Haversine underestimates true road distance by ~20–30%.	Systematically underestimates travel time; mitigated by conservative speed (20 mph) and sensitivity analysis.
A-4	Constant average EMS speed within each TOD band (20 mph base).	NYC DOT reports urban speeds 12–25 mph; 20 mph is a reasonable central estimate with lights/sirens.	Ignores route-specific congestion; partially mitigated by TOD factors.
A-5	Service time i.i.d. LogNormal(mean=25, std=10 min) for all incidents.	Literature-supported; captures right-skew and positive support.	Severity-dependent service times are ignored; sensitivity analysis on μ_s and σ_s .
A-6	Incident locations are precinct centroids (not exact crash coordinates).	30-precinct spatial resolution balances fidelity and tractability.	Intra-precinct spatial variation is lost; acceptable for strategic firehouse-level decisions.
A-7	Upon service completion, units instantaneously return to home firehouse.	Service-time distribution is calibrated to include return component.	Slight overestimate of availability during return; conservative from a policy-evaluation perspective.
A-8	No dispatch priority or preemption.	Out-of-scope; focuses on spatial staging rather than triage.	Model does not differentiate critical vs. non-critical calls.
A-9		Evaluates strategic staging; dynamic re-	

ID	Assumption	Justification	Impact if Violated
	Static allocation: unit positions fixed for the entire replication.	location is a separate problem.	Cannot capture real-time repositioning benefits.
A-10	Empty-and-idle initial conditions at $t = 0$.	Natural for a fresh deployment cycle start.	Brief transient at start; negligible over 168 h horizon.
A-11	No balking, reneging, or abandonment from the queue.	All crashes require response regardless of wait.	Queue may over-accumulate under extreme load; realistic for emergency services.
A-12	Firehouses have a maximum capacity of 5 units.	Physical space constraint from optimisation model.	May restrict optimal solutions; configurable parameter.

12.2 Limitations and Exclusions

Exclusion	Reason
Real-time dynamic relocation	Out-of-scope; this study evaluates static staging policies only.
Congestion-dependent travel times	Would require a full traffic simulation model; beyond project scope.
Full road-network routing	Requires OSRM/GraphHopper integration; Haversine proxy is a defensible simplification for strategic analysis.
Dispatch priority / preemption	All incidents treated equally; priority modelling is a future extension.
Hospital turnaround delays	Would add complexity with limited impact on firehouse staging decisions.
Non-crash EMS demand	Study focuses on motor-vehicle collision response; other call types excluded.
Multiple-unit incidents	Assumed single-unit; major incidents are rare and would require a separate protocol model.
Shift changes / breaks	Units are available 24/7 for the simulation horizon; workforce scheduling is out-of-scope.
Equipment failures / breakdowns	All units assumed to be operational throughout the replication.

12.3 Sensitivity Analysis Plan

To assess robustness of results to key assumptions, the following parameters will be varied in sensitivity experiments:

Parameter	Base Value	Sensitivity Range
Base arrival rate λ_0	3.48 /hr	$\pm 25\%$ (2.61 – 4.35)
Average EMS speed v	20 mph	15 – 25 mph
Service time mean μ_s	25 min	20 – 35 min
Total units K	40	{20, 30, 40, 48}
Response-time threshold τ	8 min	{6, 8, 10, 12} min

13. References

1. **Law, A. M.** (2015). Simulation Modeling and Analysis, 5th ed. McGraw-Hill.
Chapters 9 (terminating simulation output analysis), 12 (random variate generation).
 2. **Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M.** (2014).
Discrete-Event System Simulation, 5th ed. Pearson.
 3. **Lewis, P. A. W., & Shedler, G. S.** (1979). Simulation of nonhomogeneous
Poisson processes by thinning. *Naval Research Logistics Quarterly*, 26(3), 403–413.
 4. **Daskin, M. S.** (1983). A maximum expected covering location model.
Transportation Science, 17(1), 48–70.
 5. **Church, R. L., & ReVelle, C.** (1974). The maximal covering location problem.
Papers of the Regional Science Association, 32, 101–118.
 6. **McLay, L. A., & Mayorga, M. E.** (2010). Evaluating emergency medical
service performance measures. *Health Care Management Science*, 13(2), 124–136.
 7. **Budge, S., Ingolfsson, A., & Zerom, D.** (2010). Empirical analysis of
ambulance travel times. *Management Science*, 56(4), 716–723.
 8. **SimPy Documentation** — <https://simpy.readthedocs.io/en/latest/>
-

Document prepared as part of the EMS Readiness Optimization project, Phase 4.

Cross-references: `docs/project_charter.md` , `docs/assumptions_log.md` ,
`docs/service_model_spec.md` , `docs/optimization_formulation.md` .